ZB_ZC_RATIO · ANSWERS FROM THE LAST MEETING

# How we label jet flavour in MC

hadronFlavour and partonFlavour: where they come from, what our code does with them, and where the 99% / 95% on slide 21 comes from

Summer22 DY→$\ell\ell$ MC · 20 files per sample · 21 October 2026

# The questions, and the short answers

### Do we calculate partonFlavour ourselves?

No. Every MC jet in NanoAOD already has it (Jet_partonFlavour). We read it as it is.

### Is it the official CMS way?

Yes. CMS fills it with its own tool, the same for every analysis. We don't change any jet's value.

### Did the 99% / 95% on slide 21 come from the TWiki?

No. We measured it ourselves on our 20-file DY MC (99.3% b, 95.3% c). The TWiki gives no numbers, only that the two labels disagree "in rare instances".

### Is our b / c / uds / g grouping made up?

No. It follows the standard particle codes and is the same as the official BTV code, except that we put s together with u and d.

**Bottom line**

The flavour labels are 100% the official CMS ones. The only things that are ours are the grouping (a relabelling) and the agreement percentage (a measurement).

# What "jet flavour" means in simulation

A jet starts from one quark or gluon. In data we never know which one. In MC we do, because the generator remembers every particle it made. CMS uses that to put a "truth" label on each MC jet.

## Jet_hadronFlavour

- Looks at HADRONS (what a quark turns into: B mesons, D mesons, ...)
- Values: 5 = b, 4 = c, 0 = light
- What BTV recommends for b and c tagging
- All b/c-tag scale factors are measured for it
- This is what we use for Z+b / Z+c

## Jet_partonFlavour

- Looks at PARTONS (the quarks and gluons themselves)
- Values: 5 = b, 4 = c, 1/2/3 = d/u/s, 21 = gluon, 0 = nothing found
- Can tell gluon jets from u/d/s jets (hadronFlavour can't)
- We only use it as a cross-check

### Both labels are already in NanoAOD

Same CMS tool, same method. Only the input is different: one looks at hadrons, the other at partons.

# Which quarks belong to a jet? The "ghost" trick

Old way: draw a circle around the jet. Problem: the answer depends on the circle size. CMS dropped that. The current way has no circle at all:

### Step 1

Take the generator particles you care about (B/C hadrons, or quarks and gluons)

### Step 2

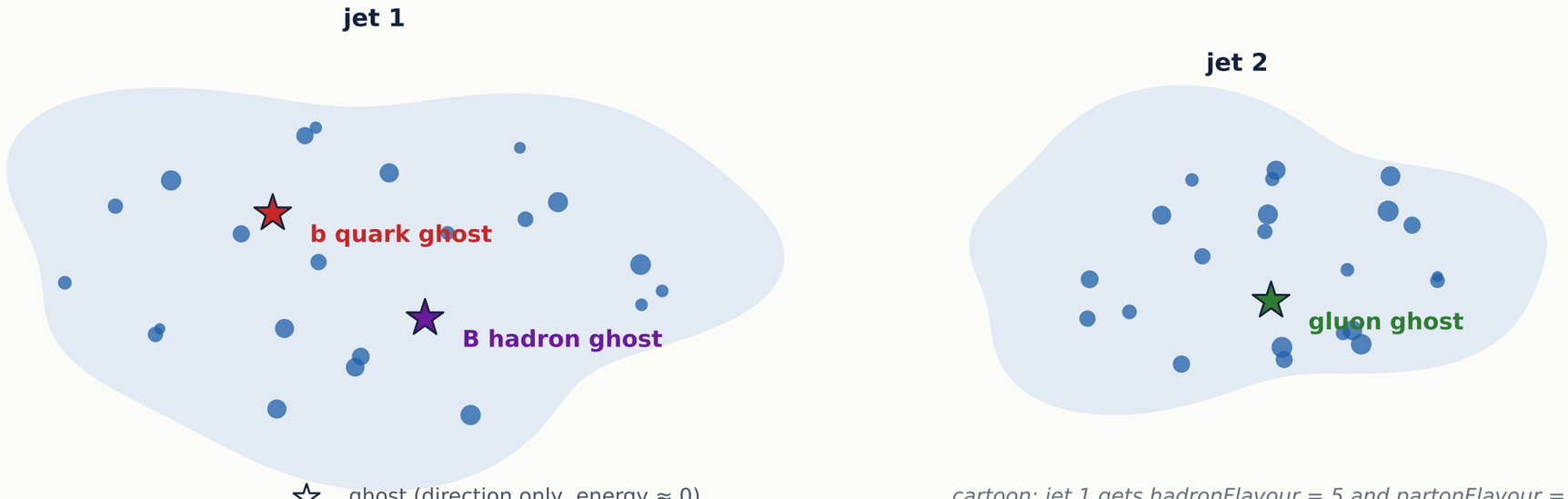
Make their energy almost zero ($\times 10^{-18}$). Direction stays the same. These are the "ghosts"

### Step 3

Build the jets again, with the ghosts mixed in with the real particles

### Step 4

See which ghosts ended up inside each jet. Ghosts carry no energy, so the jets don't change




*cartoon: jet 1 gets hadronFlavour = 5 and partonFlavour = 5; jet 2 gets 0 and 21*

Jet flavour labels · Summer22 DY→ℓℓ MC · 20 files per sample · leading AK4 jet · 4

# The rules: which number a jet gets

## hadronFlavour (from hadron ghosts)

| value | when |
|---|---|
| 5 (b) | at least one B hadron in the jet |
| 4 (c) | a C hadron, but no B hadron |
| 0 (light) | neither |

## partonFlavour (from quark/gluon ghosts)

| value | when |
|---|---|
| 5 (b) | at least one b quark in the jet |
| 4 (c) | a c quark, but no b quark |
| 1, 2, 3 | only d/u/s: the most energetic one wins |
| 21 (g) | same, but the most energetic is a gluon |
| 0 | no quark or gluon landed in the jet |

- A minus sign means antiquark (−5 = anti-b). We drop the sign.
- The two labels are worked out separately. Neither is allowed to overwrite the other (CMS setting hadronFlavourHasPriority = False).
- That is why, for a few jets, they can disagree (more on slide 11).

# Where this is written down officially

## 1. CMS TWiki: SWGuideBTagMCTools, "Hadron parton based jet flavour"

https://twiki.cern.ch/twiki/bin/view/CMSPublic/SWGuideBTagMCTools#Hadron_parton_based_jet_flavour

- Gives the rules on the previous slide
- "the main use case for the hadron-based flavour is b tagging"
- hadron flavour "cannot be used to distinguish different types of light-flavour jets" (that's what partonFlavour is for)
- "In rare instances a conflict between the hadron- and parton-based flavours can occur" (no numbers given)

## 2. The CMS software that fills the NanoAOD branch (CMSSW 13_0_X, used for Summer22)

```
PhysicsTools/NanoAOD/python/jetMC_cff.py
    partonFlavour = Var("partonFlavour()", "int16", doc="flavour from parton matching")
PhysicsTools/PatAlgos/python/mcMatchLayer0/jetFlavourId_cff.py
    patJetFlavourAssociation = JetFlavourClustering   (the ghost method)
    jetAlgorithm="AntiKt", rParam=0.4, ghostRescaling=1e-18, hadronFlavourHasPriority=False
PhysicsTools/PatAlgos/python/producersLayer1/jetProducer_cfi.py
    useLegacyJetMCFlavour = False   (old circle method only for heavy-ion running)
PhysicsTools/PatAlgos/python/slimming/miniAOD_tools.py + tools/jetTools.py
    Run 3 PUPPI jets (slimmedJetsPuppi) get their own copy of the same ghost tool
```

**Heads-up: the doc string says "parton matching". That's an old name. The value really comes from the ghost method.**

# Checked directly in one of our input files

Summer22 DY NanoAODv12 file used in the run (read with uproot, 2026-10-21):

```
/store/mc/Run3Summer22NanoAODv12/DYto2L-2Jets_MLL-50_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/
    NANOAODSIM/130X_mcRun3_2022_realistic_v5-v5/120000/02c39db3-e55f-45aa-b3e8-cca8f2fbe820.root

branch              type     description stored in the file
Jet_partonFlavour   int16    "flavour from parton matching"
Jet_hadronFlavour   uint8    "flavour from hadron ghost clustering"
nJet                int32    "slimmedJetsPuppi, i.e. ak4 PFJets Puppi with JECs applied, ..."

partonFlavour values seen:  -5 -4 -3 -2 -1  0  1  2  3  4  5  21
hadronFlavour values seen:   0  4  5
```

## What this tells us

The branches are there, they are filled by CMS, and they only take the values the rules allow. The jets are PUPPI jets, which (previous slide) use the same ghost tool.

# What our code does with these numbers

src/BTVNanoCommissioning/workflows/Zb_Zc_ratio.py  (exact code, read from the file when making this slide)

```
def jet_flavour(self, jet):
    if self.jet == "AK4":
        hf = ak.to_numpy(jet.hadronFlavour)
        return np.select([hf == 5, hf == 4], ["b", "c"], default="l")
    ...  # AK8 jets (nBHadrons / nCHadrons), not used here

@staticmethod
def parton_flavour(jet):
    pf = np.abs(ak.to_numpy(jet.partonFlavour))
    return np.select(
        [pf == 5, pf == 4, (pf >= 1) & (pf <= 3), pf == 21],
        ["b", "c", "uds", "g"],
        default="none",
    )
```

- Leading AK4 jet only: pT > 30 GeV, |η| < 2.5, jet ID, ΔR > 0.4 from both leptons
- hadronFlavour → b / c / light. This is the label for the Z+b / Z+c result
- partonFlavour → drop the sign, then b / c / uds / g / none
- No circles, no matching, no thresholds of our own
- We never change a jet's value, we only sort it into groups

# Is our grouping made up? No

The numbers are standard particle codes (PDG IDs), used by CMS and every generator: 1 = d, 2 = u, 3 = s, 4 = c, 5 = b, 21 = gluon. The official BTVNanoCommissioning code groups them almost the same way:

```
# upstream cms-btv-pog master:
# utils/histogramming/histograms/qgtag.py
def _flavor_label(flav):
    absflavs = np.abs(flav)
    # 1,2 -> ud   3 -> s   4 -> c
    # 5 -> b   21 -> g   else -> other
```

| \|partonFlavour\| | official BTV | ours |
|---|---|---|
| 5 | b | b |
| 4 | c | c |
| 1, 2 | ud | uds |
| 3 | s | uds |
| 21 | g | g |
| 0 | other | none |

**Only difference: s is kept on its own in BTV, merged with u/d by us**

The slide-21 numbers only use the b and c groups, which are identical to BTV's. The s merge only affects the light-jet split (uds vs g). Easy to split back out if we prefer the BTV convention. Other official workflows (DY_sfl.py, histogrammer.py) also keep partonFlavour = 0 as its own class.

# Our numbers: how often do the two labels agree?

Start from the hadronFlavour label (what we use), then ask what partonFlavour says. Z→ee + Z→μμ, weighted by cross-section (what slide 21 shows).

| | MC events | parton b | parton c | parton uds | parton g | parton none |
|---|---|---|---|---|---|---|
| hadron b | 82,303 | 99.31% | 0.13% | 0.15% | 0.42% | 0.00% |
| hadron c | 151,037 | 0.10% | 95.27% | 2.81% | 1.82% | 0.01% |
| hadron light | 1,585,032 | 0.15% | 1.03% | 67.01% | 31.31% | 0.50% |

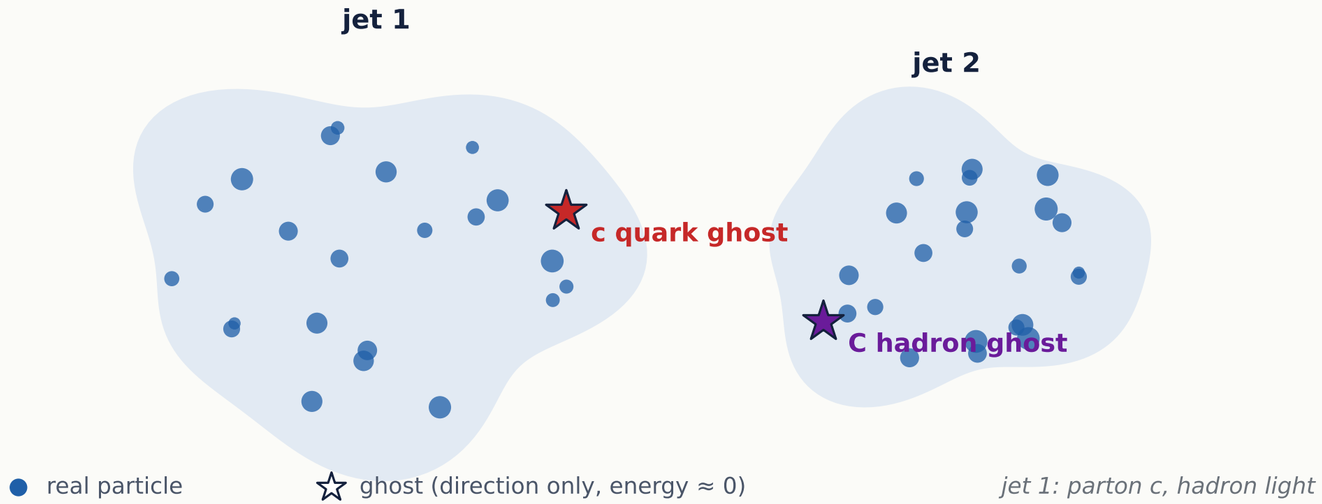
## Checked three ways, all agree

- Weighted, jet_pt histogram (the slide): b 99.3%, c 95.3%
- Plain event count (nevt_migration): b 99.4%, c 94.2%
- Per channel, c: Z→ee 95.5%, Z→μμ 95.1%

## Why the old deck said 92% for c

- The 1-file-per-sample deck had few events: c came out 92.4%
- Z→μμ alone was 90.5%
- With 20 files it settles at 95%. Use the 20-file number

# Why the labels can disagree + a warning




- A quark and the hadron it becomes don't fly in exactly the same direction
- Near a jet's edge, the quark ghost can land in one jet and the hadron ghost in the next one (or in no jet)
- Then the labels differ. Normal for the method, not a bug in our code

## Careful: the 95% only works in one direction

"Of jets that hadronFlavour calls c, 95% are partonFlavour c": yes. But "of jets that partonFlavour calls c, how many are hadronFlavour c?" gives only 90%. Reason: 1.0% of light jets carry a c quark ghost, and light jets are so common that this is 10% of all partonFlavour-c jets. (Same for b: 97% the other way.) The slide wording "of b jets / of c jets" means hadron-flavour jets, so it's correct as written.

# Summary

• hadronFlavour and partonFlavour both come straight from NanoAOD. CMS fills them with its official ghost tool. We don't compute or change them.

• We use hadronFlavour for the result, as BTV recommends. partonFlavour is only a cross-check (and splits light jets into uds vs gluon).

• Our grouping is the standard particle codes, same as the official BTV code (except s is merged into uds).

• The 99% (b) and 95% (c) agreement is our own measurement on 20 files/sample. CMS gives no official number. The TWiki just says disagreements are "rare", which fits.

• Full write-up: NOTES_partonFlavour_explained.txt in the repo.

**Open question for the group**

Do we want to split s back out of uds, so the light-jet plots match the BTV convention exactly? It doesn't change any b or c number.